

Computer Vision Homework 3

Name: NICKHOLAS

Student ID: M11515801

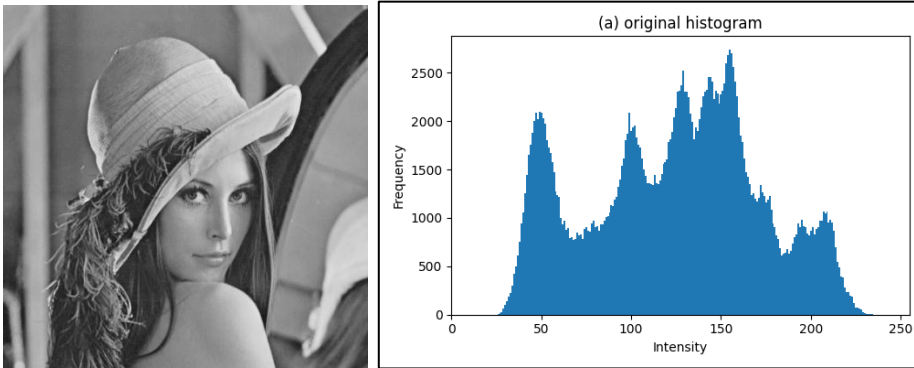
Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement all the image processing and histogram requirements.

Below are the results and brief explanations

(a) original image and its histogram



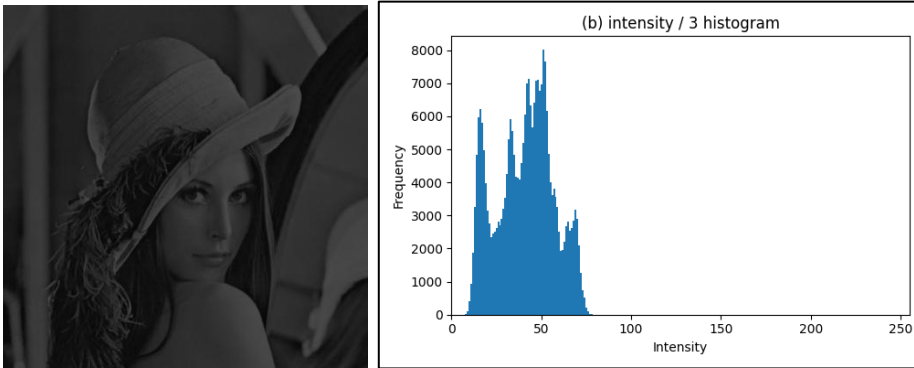
Explanation:

Code below is defined to compute, save, produce histogram and the histogram graph image (this function is used to produce histogram for part b and c too). When the histogram is computed, an integer array is used to count the numbers of each grey level. Each level value corresponds to the index of each element.

```
def compute_histogram(img):  
    hist = [0] * 256  
  
    for row in img:  
        for pixel in row:  
            hist[pixel] += 1  
  
    return hist
```

```
def save_histogram(hist, name, title):  
    # excel spreadsheet  
    wb = Workbook()  
    ws = wb.active  
    ws.title = "Histogram"  
  
    ws.append(["Intensity", "Count"])  
    for i, val in enumerate(hist):  
        ws.append([i, val])  
  
    wb.save(name + ".xlsx")  
  
    # histogram image  
    fig = plt.figure(figsize=(6, 4), dpi=100)  
    plt.bar(range(256), hist, width=1.0)  
    plt.title(title)  
    plt.xlabel("Intensity")  
    plt.ylabel("Frequency")  
    plt.xlim(0, 255)  
    plt.tight_layout()  
    fig.canvas.draw()  
    rgba = np.asarray(fig.canvas.buffer_rgba())  
    plt.close(fig)  
    cv2.imwrite(name + ".bmp", cv2.cvtColor(rgba, cv2.COLOR_RGBA2BGR))
```

(b) image with intensity divided by 3 and its histogram

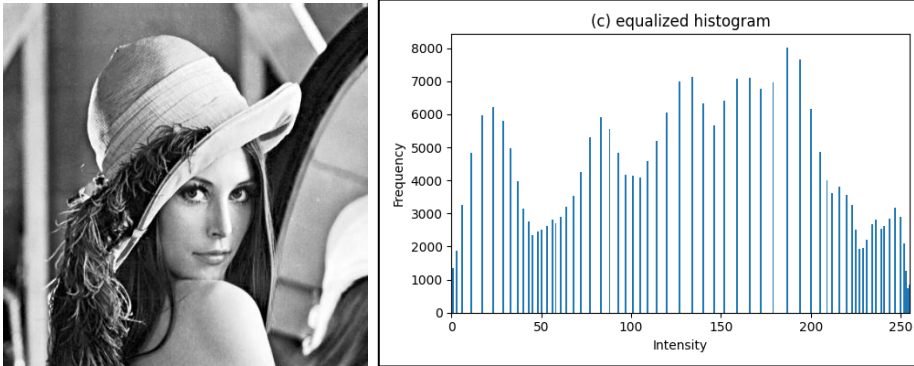


Explanation:

```
def divide_by_3(img):  
    h, w = img.shape  
    out = np.zeros((h, w), dtype=np.uint8)  
  
    for i in range(h):  
        for j in range(w):  
            out[i][j] = int(img[i][j]) // 3  
  
    return out
```

In this part, the original image intensity is scaled down by 3 using integer division, so the maximum value is changed from 255 to 85, all values fall into the range of 0 to 85 and resulting in a darker image with lower contrast. After scaling the intensity, then run the same process using function that is mentioned in part (a) to produce the histogram. After seeing the histogram result, we can see that the bars are concentrated on the left side of the graph (0-85 intensity) and the intensities from 86 onwards have a count of zero.

(c) image after applying histogram equalization to (b) and its histogram



Explanation:

```
def equalize(img, hist):
    h, w = img.shape
    total = h * w

    cdf = np.cumsum(hist)
    cdf_min = 0
    for v in range(256):
        if hist[v] != 0:
            cdf_min = cdf[v]
            break

    lut = [0] * 256
    for v in range(256):
        if total == cdf_min:
            lut[v] = v
        else:
            lut[v] = int((cdf[v] - cdf_min) / (total - cdf_min) * 255 + 0.5)
            lut[v] = max(0, min(255, lut[v]))

    out = np.zeros((h, w), dtype=np.uint8)
    for i in range(h):
        for j in range(w):
            out[i][j] = lut[img[i][j]]

    return out
```

The histogram equalization mapping function is derived from the histogram of (b). The histogram is accumulated into a Cumulative Distribution Function (CDF), which is then normalized to the range 0 to 255 to form a lookup table, and each pixel is mapped to its new intensity using this table. As a result, the image becomes brighter with higher contrast, and its histogram is spread across the whole 0 to 255 range instead of only 0 to 85. Then, run the same process using function that is mentioned in part (a) to produce the histogram.